

DOCUMENTATION TECHNIQUE

Dashboard web iot-frontend

Application React/Vite de supervision : métriques en temps réel, état du boîtier, analyses IA, caméra et reconnaissance faciale, paramètres de notification.

Projet	Sentinel-X — Workshop EPSI Bac+4, consortium AetherCorp
Pile	React 19, TypeScript, Vite, client Better Auth, SVG natif, nginx en production
Public	Développeurs frontend et jury technique
Version	Octobre 2026 — état du dépôt à la date de génération

Sommaire

- 01** Rôle et principes
Un client léger qui ne parle qu'au backend
 - 02** Architecture
Domaine, ports, adaptateurs, présentation
 - 03** Flux de données
Chargement initial, SSE, sondages, reconnexion
 - 04** Écrans et composants
Métriques, Caméra, Paramètres, Connexion
 - 05** Règles d'affichage
Fraîcheur, états inconnus, simulation
 - 06** Interface et accessibilité
Thème, responsive, graphiques
 - 07** Sécurité
Session, CSP, aucun secret côté client
 - 08** Build, déploiement et tests
Vite, nginx, Docker, contrôles
-

1 Rôle et principes

`iot-frontend` est le dashboard de supervision des opérateurs. Il affiche en temps réel les mesures et l'état du boîtier, les analyses du service IA et la webcam du serveur. Il permet aussi de gérer la reconnaissance faciale et de régler les notifications.

- **Un seul interlocuteur** : l'application n'appelle que `/api/*` sur la même origine. Elle ne contacte jamais MQTT, FastAPI ni la caméra directement.
- **Aucune donnée inventée** : une mesure absente, périmée ou invalide s'affiche comme inconnue, jamais comme un zéro ou un état normal.
- **Pas de dépendance graphique lourde** : graphiques en SVG natif, icônes en ligne, aucune bibliothèque de composants.



Figure 1. Le dashboard ne connaît que le backend, servi sur la même origine.

2 Architecture

L'application suit la même **clean architecture** que le backend. Les composants React ne connaissent que des **ports**, c'est-à-dire des interfaces. Les implémentations concrètes sont instanciées une seule fois dans `main.tsx` et injectées par un contexte React (`ServicesContext`). Remplacer le transport (REST, SSE) ou l'authentification ne touche donc pas à la présentation.

Couche	Dossier	Contenu
Domaine	<code>src/domain</code>	Types et règles pures : fusion et bornage de l'historique (<code>telemetry.ts</code>), fraîcheur de l'état du boîtier (<code>esp-status.ts</code>), priorité réel/simulation des analyses (<code>ai.ts</code>), détections et visages (<code>vision.ts</code> , <code>faces.ts</code>), validation de l'enrôlement (<code>face-enrollment.ts</code>), notifications, session
Application	<code>src/application</code>	Ports <code>DeviceApi</code> , <code>AiApi</code> , <code>SettingsApi</code> , <code>LiveFeed</code> , <code>AuthService</code> , <code>ThemeStore</code> ; erreurs (<code>UnauthorizedError</code>) ; cas d'usage <code>startVisionAndWait</code>
Infrastructure	<code>src/infrastructure</code>	Client HTTP commun (<code>http-client.ts</code>), adaptateurs <code>HttpDeviceApi</code> , <code>HttpAiApi</code> , <code>HttpSettingsApi</code> , <code>SseLiveFeed</code> (<code>EventSource</code>), <code>BetterAuthService</code> , <code>LocalThemeStore</code> , préparation des photos (<code>face-enrollment-payload.ts</code>) et URL du flux (<code>camera-stream.ts</code>)
Présentation	<code>src/presentation</code>	<code>App.tsx</code> , pages (<code>MetricsPage</code> , <code>CameraPage</code> , <code>SettingsPage</code>), composants, hooks (<code>useSession</code> , <code>useDeviceStatus</code> , <code>useLiveData</code> , <code>useNotificationSettings</code> , <code>useRoute</code> , <code>useTheme</code>), styles partagés <code>index.css</code>
Composition	<code>src/main.tsx</code>	Instanciation des adaptateurs et rendu de l'application

Le routage utilise le fragment d'URL (`#/metrics`, `#/camera`, `#/settings`), ce qui fonctionne derrière n'importe quel serveur statique sans règle de réécriture.

3 Flux de données

Donnée	Mécanisme	Fréquence
Session	<code>AuthService.getSession()</code> au démarrage, puis à chaque 401	à la demande
État MQTT et dernière mesure physique	<code>GET /api/status</code>	toutes les 2 s
Historique des mesures	<code>GET /api/readings?limit=150</code> puis événements SSE <code>reading</code>	initial + temps réel
Analyses IA	<code>/api/ai/history</code> et <code>/api/ai/latest</code> , puis SSE <code>ai</code>	initial + temps réel
Disponibilité IA, webcam, visages	SSE <code>ai-status</code> et <code>GET /api/ai/status</code>	temps réel + toutes les 5 s
Détections vision externes	<code>GET /api/vision</code> puis SSE <code>vision</code>	initial + temps réel
Vidéo	Balise image sur <code>/api/camera/stream</code> (MJPEG)	flux continu

- **Un seul EventSource** (`/api/stream`) transporte les quatre types d'événements.
- **Fusion sans doublon** : l'historique et le direct sont fusionnés par identifiant, triés par heure de mesure et bornés à 150 mesures (environ 5 minutes) et 50 analyses. Une mesure arrivée en retard prend sa place chronologique.
- **Reconnexion** : après une coupure réseau, l'historique est rechargé pour récupérer les événements manqués. Si le serveur ferme le flux (session expirée, redémarrage), la session est vérifiée et le flux rouvert après 5 s.
- **Pas de régression** : une réponse HTTP plus ancienne ne remplace jamais un état SSE plus récent.
- **Démarrage de la webcam** : `startVisionAndWait` envoie `start`, puis interroge `/api/ai/vision/status` chaque seconde (20 s maximum) jusqu'à l'état `running`. Le flux vidéo n'est ouvert qu'à ce moment, avec une URL unique à chaque tentative.

4 Écrans et composants

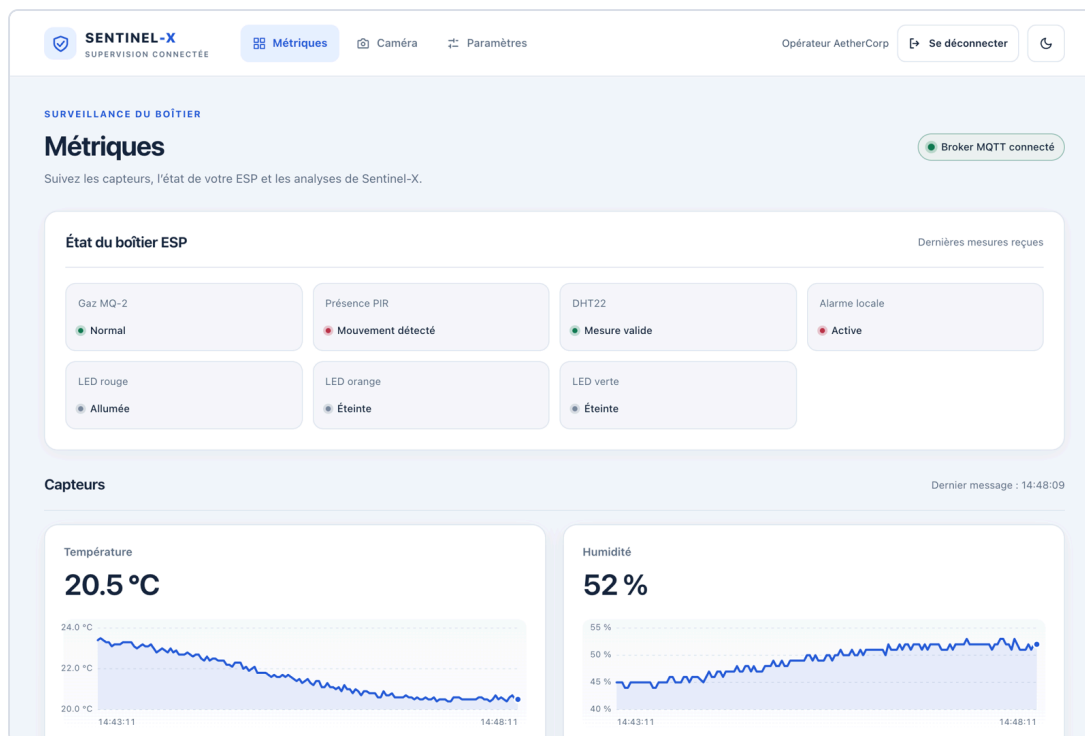


Figure 2. Page Métriques : barre de navigation, indicateur MQTT, état du boîtier, cartes capteurs.

Écran	Composants	Rôle
Toutes les pages	Topbar	Navigation, utilisateur connecté, déconnexion, bascule clair/sombre
Connexion	LoginForm	E-mail et mot de passe ; messages pour identifiants invalides et blocage temporaire
Métriques	StatusIndicator	Pastille verte/rouge/grise de l'état du broker
	EspStatusPanel	Gaz, PIR, DHT, alarme locale et LEDs remontés par le firmware
	LineChart + tableaux	Valeur courante, graphique temps réel et 20 dernières mesures par capteur
	AiPanel, VisionCamera	AI ENGINE : anomalie, risque, confiance, raisons, webcam et événements récents
	Commande LED	Boutons ON/OFF (nécessitent un firmware MQTT compatible)
Caméra	CameraFeed, VisionControls	Flux vidéo, démarrage/arrêt de la webcam, réessai
	FaceRecognitionPanel	Visages courants, catalogue, rechargement, historique des reconnaissances
	FaceEnrollmentForm	Ajout d'une personne : nom, 1 à 5 photos, aperçus locaux, bilan photo par photo
Paramètres	SettingsPage	Adresse de réception, activation, types d'alertes, e-mail de test

5 Règles d'affichage

- **État du boîtier** : sans mesure physique depuis 30 s, tous les états passent à « Inconnu ». Les indicateurs absents (préchauffage, capteur en erreur) restent inconnus.
- **Graphiques** : une interruption de plus de 10 s entre deux mesures coupe la courbe au lieu de relier les points ; une mesure isolée est affichée comme un point.
- **Analyses IA** : une analyse réelle récente (moins de 30 s) est prioritaire sur une analyse simulée. Les données de démonstration portent la mention **SIMULATION**. Une couverture incomplète affiche « Analyse partielle » et un niveau **INDÉTERMINÉ** plutôt que SAFE.
- **Vision** : une détection de plus de 5 s n'est plus présentée comme courante ; un visage inconnu est signalé.
- **Erreurs** : un 401 renvoie à l'écran de connexion ; les messages du serveur sont affichés tels quels dans la zone concernée.

6 Interface et accessibilité

- **Thème** : clair ou sombre, il suit le système jusqu'au premier choix, puis le mémorise (`localStorage`). `public/theme-init.js` l'applique avant le premier affichage pour éviter un flash. C'est un fichier externe, pour rester compatible avec la CSP.
- **Couleurs** : jetons CSS communs (`index.css`) pour les deux thèmes ; une couleur d'état est toujours accompagnée d'un texte, jamais seule.
- **Responsive** : grilles adaptatives jusqu'à 320 px de large ; les tableaux larges défilent dans leur propre conteneur.
- **Accessibilité** : focus clavier visible, libellés ARIA sur les boutons à icône, légendes cachées sur les tableaux, zones `aria-live` pour les messages, animations désactivées si le système le demande.
- **Graphiques** : SVG avec ligne de 2 px, aplats légers, grille discrète, infobulle et réticule au survol ; le tableau des dernières mesures sert d'alternative textuelle.



Figure 3. Carte capteur : valeur courante, graphique temps réel et dernières mesures.

7 Sécurité

- La session est un cookie `HttpOnly` géré par Better Auth : il n'est pas lisible par le JavaScript de la page.
- Aucun secret ni URL interne (FastAPI, broker, jeton IA) n'est présent dans le code client.
- En production, nginx applique une Content Security Policy stricte (`default-src 'self'`, `frame-ancestors 'none'`), `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff` et `Referrer-Policy: no-referrer`.
- Les photos d'inscription sont vérifiées côté client (format, taille, nombre) pour un retour immédiat. Le serveur refait toutes les vérifications, qui seules font foi.

8 Build, déploiement et tests

```
npm run dev      # Vite, /api relayé vers http://127.0.0.1:3001
npm test        # règles du domaine et adaptateurs (runner natif de Node)
npm run lint    # oxlint
npm run build   # tsc -b puis vite build vers dist/
```

Production : l'image Docker compile l'application puis la sert avec `nginx-unprivileged` (utilisateur non-root, port 8080). nginx relaie `/api/` vers le backend sur le réseau interne, avec une limite de taille élargie uniquement pour la route d'inscription facial.

Vérification (état du dépôt à la génération)	Résultat
<code>npm test</code> — domaine, flux SSE, IA, visages, inscription, démarrage webcam	39/39 réussis
<code>npm run lint</code>	aucun avertissement

Vérification (état du dépôt à la génération)	Résultat
<code>npm run build</code>	réussi

`node scripts/check-frontend-style.mjs` pilote un navigateur sur une instance Vite isolée, avec des réponses d'API factices. Il contrôle les pages en clair et en sombre, à 1440, 375 et 320 px, le focus clavier et les contrastes. La CI GitHub Actions exécute tests, lint et build sur Node 24.